



Università Degli Studi di Salerno
Ingegneria, Gestione ed Evoluzione del Software

ReFAIR

Toward a Context-Aware Recommender
for Fairness Requirements Engineering

Master test plan

Riferimento	Report
Versione	1.1
Data	28/12/2024
Presentato da	PASTORE Carmine LAURINO Nicola



Sommario

1. Introduzione.....	3
2. Testing di unità	3
3. Test d'integrazione.....	3
4. Testing di sistema	3
5. Test di regressione	4



1. Introduzione

Il presente **Master Test Plan** descrive la strategia complessiva, gli obiettivi e l'approccio che saranno adottati per l'esecuzione delle attività di test nel contesto del progetto in corso.

In questo documento andremo a definire le tecniche di testing adottate ed i tools utilizzati per automatizzare il processo. In particolare, saranno eseguiti:

- Testing di unità
- Testing di integrità
- Testing di sistema
- Testing di regressione

Per garantire una copertura completa e un controllo più approfondito, verrà sfruttata la **test suite** già presente come base di verifica e, al contempo, verranno sviluppati nuovi casi di test dedicati a funzionalità critiche o recentemente introdotte.

2. Testing di unità

Con il testing di unità andiamo a verificare la correttezza e la robustezza delle singole componenti, in modo da identificare eventuali difetti nella logica interna del codice. Sarà adottata una strategia **white-box** utilizzando il framework **pytest**. Per garantire una visione chiara della copertura del codice utilizziamo **pytest-cov** in abbinamento a **pytest**. In questo modo, riusciamo ad ottenere la branch coverage. Ciò ci aiuta a migliorare la qualità dei test.

3. Test d'integrazione

Con il test di integrazione andiamo a verificare che i vari moduli e componenti del sistema funzionino correttamente quando vengono combinati. La strategia scelta per il testing d'integrazione è la strategia bottom-up, si comincia a testare dai moduli di basso livello, per poi salire ai moduli di livello superiore. Questo approccio permette di testare gradualmente i componenti, semplificando l'identificazione e la risoluzione dei problemi. Il tool utilizzato per questa fase di testing sarà **pytest**.

4. Testing di sistema

Con il testing di sistema andremo a valutare il funzionamento dell'intero sistema in un ambiente il più possibile simile a quello di produzione, per verificare che il software rispetti i requisiti funzionali e non funzionali. Sarà utilizzata una strategia black-box e per garantire una completezza nel testing di sistema andremo ad utilizzare la strategia Category partition, andando a considerare quelle che sono le funzionalità principali del sistema ovvero:



- Load
- Report
- Download
- Single US
- Feedback (nuova funzionalità aggiunta)
- Retraining (nuova funzionalità aggiunta)

Per l'automatizzazione del testing di sistema utilizzeremo Selenium.

5. Test di regressione

Il **Test di Regressione** ha l'obiettivo di accertare che eventuali modifiche, correzioni di bug o nuove funzionalità introdotte nel software non abbiano effetti indesiderati su aree già testate e precedentemente funzionanti. Per raggiungere questo scopo, vengono rieseguiti i test significativi su tutte le parti del sistema potenzialmente influenzate dai cambiamenti. In particolare, andremo ad utilizzare una strategia di regression testing, detta "Test all", con la quale andiamo a rieseguire tutti i casi di test sull'intero sistema ogni volta che viene introdotta una modifica o risolto un bug. Questo approccio, sebbene possa risultare più dispendioso in termini di tempo e risorse, assicura la massima copertura di test e riduce il rischio di trascurare eventuali aree impattate in modo indiretto. Questo processo sarà automatizzato attraverso le github Actions, quando si fa la push sul branch dev, un workflow di GitHub Actions avvierà in automatico l'intera suite di test. Tale automazione riduce il rischio di rilasciare modifiche non testate o che possano creare problemi a componenti non coinvolte in modo diretto.